기계화 명세 기반
이더리움 합의 클라이언트 비교 테스팅

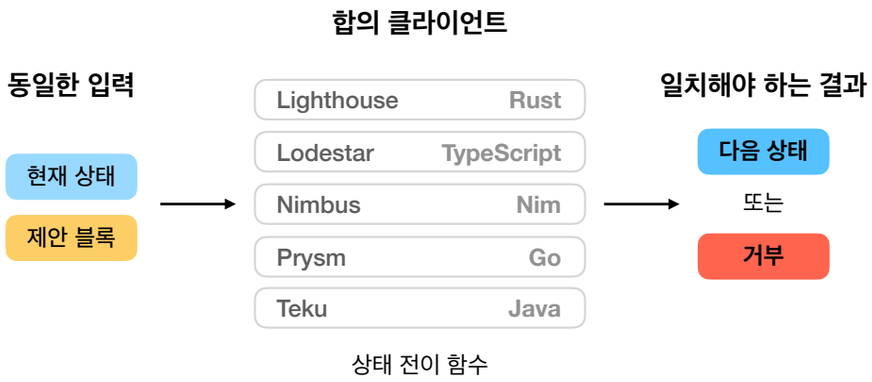
SpecTrum: Specification-Guided Differential Fuzzing for Ethereum Consensus Clients

정석훈, 단경민, 류석영, 황성재 (ASE'26)

안녕하세요. 류석영 교수님 연구실 석사과정 정석훈입니다. 지금부터 기계화 명세 기반 이더리움 합의 클라이언트 비교 테스팅 주제로 발표드리겠습니다. 해당 연구는 저희 지도교수님, 그리고 성균관대 황성재 교수님, 단경민님과 함께했으며, 올해 ASE에서 발표예정입니다. (20s)

서로 다른 클라이언트가 같은 결과를 내야 합니다

클라이언트 다양성



이더리움에서는 서로 다른 클라이언트가 같은 입력에 대해 같은 결과를 내야 합니다.

이더리움 합의 계층은 네트워크의 각 구성원이 어떤 체인을 따르고 어느 블럭을 언제 확정할지 결정합니다. 이 알고리즘의 구현체를 합의 클라이언트라고 합니다.
하나의 구현체에 있는 버그가 네트워크 전체에 영향을 줄 위험을 줄이기 위해서
여러 팀이 서로 다른 언어로 개발한 클라이언트를 한 네트워크 위에서 사용하는데, 이를 클라이언트 다양성이라고 합니다. (30s)

저희가 다루는 합의 계층에서, 각 클라이언트는 현재 상태와 제안된 블록을 받아 상태 전이 함수를 실행합니다. 이때 블록을 받아들일지 거부할지가 같아야 하고, 받아들였다면 계산한 다음 상태도 같아야 합니다.

결과가 다르면 클라이언트마다 서로 다른 블록이나 상태를 올바르다고 판단할 수 있습니다. 이는 네트워크를 분열시키거나 블록 확정의 중단으로 이어질 수 있습니다.(60s)

공식 명세: 실행 가능한 Python 코드

consensus-spec

Beacon chain state transition function

The post-state corresponding to a pre-state `state` and a signed block `signed_block` is defined as `state_transition(state, signed_block)`. State transitions that trigger an unhandled exception (e.g. a failed `assert` or an out-of-range list access) are considered invalid. State transitions that cause a `uint64` overflow or underflow are also considered invalid.

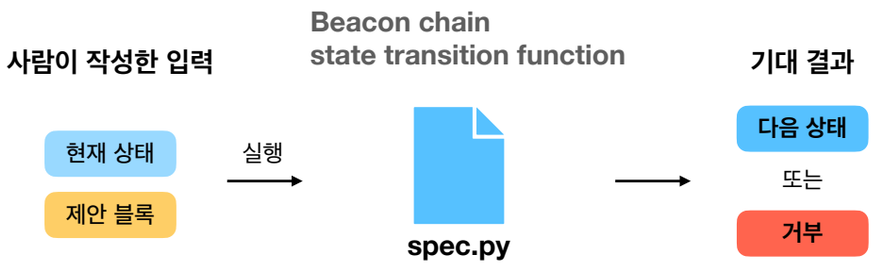
```
def state_transition(
    state: BeaconState, signed_block: SignedBeaconBlock, validate_result: b
) -> None:
    block = signed_block.message
    # Process slots (including those with no blocks) since block
    process_slots(state, block.slot)
    # Verify signature
    if validate_result:
        assert verify_block_signature(state, signed_block)
    # Process block
    process_block(state, block)
    # Verify state root
    if validate_result:
        assert block.state_root == hash_tree_root(state)
```


여러 구현체가 같은 결과를 내려면 어떤 공통의 기준, 즉 명세가 필요할텐데요.
이더리움의 공식 합의 명세인 consensus-spec은 실행 가능한 Python 코드로 제공됩니다.

화면처럼 마크다운 문서 안에 설명과 Python 코드가 함께 들어 있습니다. 개발자는 이 명세를 읽고 해당하는 언어로 클라이언트를 구현합니다. (20s)

공식 테스트: 사람이 작성

spectests



또한 이 코드를 직접 실행해, 특정 입력에서 어떤 결과가 나와야 하는지 확인할 수 있습니다. 상태 전이를 실행하면 다음 상태를 얻거나 예외가 발생합니다. 명세에서 예외는 해당 입력을 거부해야 한다는 뜻입니다.

공식 테스트인 spectest도 이 실행 가능한 명세를 활용합니다. 사람이 테스트할 입력을 작성하면, 명세를 실행해 기대 결과를 구합니다. 각 클라이언트는 같은 입력을 실행한 뒤, 자신의 결과가 기대 결과와 일치하는지 확인합니다.

즉, 공식 명세는 구현의 기준이면서 테스트의 기대 결과를 계산하는 기준이기도 합니다. (35s)

숨어있는 조건

명시적인 예외 조건

```
assert state.slot < slot
```

조건이 거짓이면 거부

숨어있는 예외 조건

```
if prev_bal * 3 >= total_bal * 2:
```

곱셈이 overflow를 일으켜도 거부

문제는 입력을 거부해야 하는 조건이 코드에 모두 명시되어 있지는 않다는 것입니다.
왼쪽의 assert에는 반드시 만족해야 하는 조건이 적혀 있습니다. 이 조건이 거짓이면 입력을 거부합니다. 따라서 코드를 읽으면 무엇을 검사해야 하는지 바로 알 수 있고, 개발자는 이 검사를 구현체에 구현하고, 이 조건을 거짓으로 만드는 테스트를 생성할 수 있습니다.
오른쪽에도 입력을 거부해야 하는 경우가 있습니다. 두 값을 비교하려면 먼저 곱셈을 계산해야 하는데, 이 명세에서는 uint64 연산의 결과가 64비트 범위를 벗어나면 예외가 발생하고 입력을 거부합니다.
그런데 코드에 적힌 비교식은 두 계산 결과의 크기를 비교할 뿐입니다. 곱셈을 범위 안에서 계산할 수 있는지를 확인하는 조건은 따로 적혀 있지 않습니다. 입력의 거부를 결정하는 조건이 연산과 정수 표현형 안에 숨어 있는 것입니다. (55s)

숨어있는 조건 → 테스트 공백

명시적인 예외 조건

```
assert state.slot < slot
```

조건을 위반하면 거부

숨어있는 예외 조건

```
if prev_bal * 3 >= total_bal * 2:
```

곱셈이 overflow를 일으켜도 거부

	분기 커버리지
조건이 참인 경우	O
조건이 거짓인 경우	O
overflow가 일어난 경우	**?**

이런 조건은 테스트에서 빠져 있어도 분기 커버리지에 드러나지 않습니다.

이 조건이 참인 경우와 거짓인 경우를 모두 테스트했다고 해 보겠습니다. 분기 커버리지에서는 비교문의 두 분기를 모두 검사했다고 표시됩니다.
하지만 두 경우 모두 곱셈이 정상적으로 계산되는 값만 사용했을 수 있습니다. 그러면 비교 결과에 따른 두 분기는 검사했어도, overflow가 발생했을 때 입력을 제대로 거부하는지는 확인하지 않은 것입니다.
이 공백은 커버리지에 미달성 항목으로도 나타나지 않습니다. 곱셈의 overflow 조건 자체가 분기 커버리지의 측정 대상에 없기 때문입니다.
따라서 이런 테스트 공백을 찾으려면, 작성자가 연산이 언제 실패하는지 직접 파악하고, 기존 테스트가 그 경우를 검사했는지도 별도로 확인해야 합니다. (50s)

숨어있는 조건 → 테스트 공백 → 미검출 불일치

명시적인 예외 조건

```
assert state.slot < slot
```

조건을 위반하면 거부

입력의 잔액 필드 하나를 변경

```
effective_bal → 2^64 - 1
```

거부	수락
Python 명세 + 구현체 2개	구현체 3개

숨어있는 예외 조건

```
if prev_bal * 3 >= total_bal * 2:
```

곱셈이 overflow를 일으켜도 거부

	분기 커버리지
조건이 참인 경우	O
조건이 거짓인 경우	O
overflow가 일어난 경우	**?**

저희는 이런 테스트 공백에서 실제 클라이언트 간 불일치를 발견했습니다.
입력의 잔액 필드 하나를 uint64 최댓값으로 바꿔 실행해 봤습니다. 같은 입력에 대해 명세는 거부했지만, 다섯 클라이언트 중 세 개는 입력을 받아들였습니다.

기존 테스트가 검사하지 않은 예외 상황에서, 입력을 거부할지 받아들일지가 서로 달랐던 것입니다. 숨어 있는 조건을 놓치면 이런 구현 차이도 발견하지 못할 수 있습니다.

따라서, 이런 불일치를 체계적으로 찾으려면, 숨어 있는 예외조건까지 테스트 대상으로 다룰 수 있어야 합니다. (35s)

명세 풀어쓰기: ① 예외 조건을 명시적으로 검사

consensus-spec (Python): weigh_and_justify_finalization (simplified)

```python
if prev_bal * 3 >= total_bal * 2:
  state.curr_jcp = prev_epoch
if curr_bal * 3 >= total_bal * 2:
  state.curr_jcp = curr_epoch
```

Consensus-SpecTec: WeighJustificationAndFinalization (simplified)

```
rule WeighJustAndFin/justify_prev:
  state, total_bal, prev_bal, curr_bal
    ~> state[.CURR_JCP = prev_epoch]
-- if total_bal <= UINT64_MAX/2
-- if prev_bal * 3 >= total_bal * 2
-- if ~(curr_bal * 3 >= total_bal * 2)
```

조건이 참	조건이 거짓
곱셈이 범위 안에 들어옴	overflow이므로 거부

저희의 해결 방법은 명세를 다시 작성해, 동작을 결정짓는 모든 조건을 명시적인 검사 대상으로 만드는 것입니다.

두 부분으로 나뉘는데요, 첫 번째 과정은 예외조건을 명시화하는 것입니다.
앞섭 본 비교문이 포함된 함수입니다. 여기서 total_bal은 uint64라는 64비트타입인데요, 이 값의 두 배가 범위를 초과하지 않으려면, 원래 값이 uint64 최댓값의 절반 이하여야 합니다. 저희는 명세를 다시 쓰면서 이 조건을 계산 규칙에 추가했습니다.

이때 추가한 조건이 참이면 곱셈 결과가 범위 안에 있고, 거짓이면 overflow가 발생하므로 입력을 거부합니다.
공식 명세에서 곱셈을 실행하다가 예외가 일어나면 입력을 거부해야 하는데, 재작성한 명세에서는 그 거부 과정이 하나의 조건으로 드러납니다.

배열 접근이나 나눗셈에도 같은 원리를 적용하여 인덱스가 범위 안에 있는지, 분모가 0이 아닌지를 조건으로 적습니다.

이렇게 하면 각 예외 조건마다 테스트에서 참과 거짓으로 평가된 적이 있는지 기록할 수 있습니다. (65s)

명세 풀어쓰기: ② 결과를 결정하는 조건의 조합 명시

consensus-spec (Python): weigh_and_justify_finalization (simplified)

```
1 if prev_bal * 3 >= total_bal * 2:  ◀ 조건 1
2   state.curr_jcp = prev_epoch
3 if curr_bal * 3 >= total_bal * 2:  ◀ 조건 2
4   state.curr_jcp = curr_epoch
```

조건 1	조건 2	최종 curr_jcp
참 또는 거짓	참	curr_epoch
참	거짓	prev_epoch
거짓	거짓	기존 값

두 번째 과정은 최종 결과를 결정하는 조건의 조합을 명시하는 것입니다.
왼쪽 코드에는 같은 필드를 갱신하는 두 개의 if문이 있습니다. 조건 1이 참이면 prev_epoch를 저장합니다. 그런데 조건 2도 참이면, 방금 저장한 값을 cur_epoch로 덮어씁니다. 따라서 최종 값이 prev_epoch이 되려면 조건 1이 참이라는 것만으로는 부족하고, 조건 2가 거짓이라는 조건도 함께 필요합니다.

이렇게 결과를 결정하는 조건의 조합을 정리하면 오른쪽처럼 세 규칙으로 나뉩니다. 조건2가 참이면 항상 cur_epoch이 남고, 뒤 조건이 거짓이고 앞 조건이 참이면 prev_epoch가 남으며, 둘 다 거짓이면 기존 값을 유지합니다. (40s)

명세 풀어쓰기: ② 결과를 결정하는 조건의 조합 명시

consensus-spec (Python): weigh_and_justify_finalization (simplified)

```python
if prev_bal * 3 >= total_bal * 2:  ◀ 조건 1
  state.curr_jcp = prev_epoch
if curr_bal * 3 >= total_bal * 2:  ◀ 조건 2
  state.curr_jcp = curr_epoch
```

Consensus-SpecTec: WeighJustificationAndFinalization (simplified)

```
rule WeighJustAndFin/justify_prev:
  state, total_bal, prev_bal, curr_bal
    ~> state[.CURR_JCP = prev_epoch]
-- if total_bal <= UINT64_MAX/2
-- if prev_bal * 3 >= total_bal * 2
-- if ~(curr_bal * 3 >= total_bal * 2)
```

조건 1	조건 2	최종 curr_jcp
참 또는 거짓	참	curr_epoch
참	거짓	prev_epoch
거짓	거짓	기존 값

예외가 일어나지 않으면,
세 규칙은 **서로소**이며 **완전**

풀어 쓴 명세에서는 이렇게 세 가지 가능한 결과를 각각 별도의 규칙으로 정의합니다. 예를 들어, 최종 결과가 prev_epoch이 나오는 경우는, 첫 번째 조건이 성립하고 두 번째 조건이 성립하지 않는다는 것을 명시해줍니다. 같은 방식으로 다른 두 경우도 각각 따로 규칙을 정의해줍니다.
예외가 일어나지 않는 경우, 이 세 규칙은 서로 겹치지 않으면서 모든 경우를 다룹니다. 즉, 서로소이면서 완전합니다.

이렇게 함으로써 여러 if문과 갱신을 순서대로 따라가며 결과를 추론하는 대신, 원하는 결과에 필요한 조건을 바로 살펴볼 수 있습니다. (35s)

핵심: 명세 풀어쓰기

동작을 결정짓는 조건들을 명시적인 검사 대상으로

① 숨어있는 예외 조건을 명시화
→ 예외로 인한 거부도 테스트 가능
② 최종 결과에 따라 조건의 조합 명시
→ 동작을 기준으로 테스트 가능

명세언어: P4-SpecTec
검증: 공식 테스트 910개에서 원본 Python 명세와 동작이 일치하는지 확인

핵심 아이디어를 정리하면, 저희는 명세의 동작을 결정하는 조건들을 명시적인 검사 대상으로 만들었습니다.
숨어있는 예외 조건을 직접 적어 테스트 대상이 될 수 있게 하고,
최종 결과에 따라 조건을 정리하여 실제 동작을 결정짓는 조건을 겨냥하기 쉽게 만들었습니다.

P4-SpecTec 명세언어를 활용했으며,
재작성한 명세는 공식 테스트 910개에서 원본 Python 명세와 동작이 일치했습니다.

이제 이 명세에서 각 조건의 참과 거짓을 측정해, 기존 테스트가 무엇을 검사하지 않았는지 살펴보겠습니다. (40s)

테스트 공백 찾기

Consensus-SpecTec: WeighJustificationAndFinalization (simplified)

```
1 rule WeighJustAndFin/justify_prev:
2   state, total_bal, prev_bal, curr_bal
3     ~> state[.CURR_JCP = prev_epoch]
4 -- if total_bal <= UINT64_MAX/2
5 -- if prev_bal * 3 >= total_bal * 2
6 -- if ~(curr_bal * 3 >= total_bal * 2)
```

공식 end-to-end test 581개의
각 `-- if` 조건별 성공/실패여부 추적

조건이 참인 입력	조건이 거짓인 입력
21	**0**
16	5
9	7

true값을 일으키는 seed를 변이해 false값을 생성

공식 spectests 581개를 재작성한 명세 위에서 실행하여 각 if 조건의 참 거짓 값이 나타난 횟수를 기록합니다.
앞서 본 overflow 방지 조건은 기존 테스트에서 참으로 21번 평가됐지만, 거짓으로는 한 번도 평가되지 않았습니다. 즉 이 조건에 도달한 테스트는 21개나 되는데 overflow가 발생하는 쪽은 한 번도 검사하지 않은 것입니다.

이처럼 한 번도 거짓으로 평가된 적 없는 조건을 찾습니다. 이 때 거짓이 될 수 없는 조건은 따로 분류해 대상에서 제외합니다.

조건의 역할에 따라 거짓의 의미도 달라집니다. 공통으로 적용되는 예외 방지 조건이 거짓이라면 입력을 거부합니다. 경우를 나누는 조건이 거짓이면 다른 규칙이 적용될 수 있습니다. 저희는 두 종류 모두 기록하고 테스트 목표로 지정합니다.

이를 통해 기존 테스트가 어떤 조건을 놓쳤는지 알 수 있습니다. 다음 단계에서는 그 조건에 도달하는 기존 입력을 출발점으로 사용합니다. (70s)

유래 추적 및 테스트 생성

Consensus-SpecTec: WeighJustificationAndFinalization (simplified)

```
rule WeighJustAndFin/justify_prev:
  state, total_bal, prev_bal, curr_bal
    ~> state[.CURR_JCP = prev_epoch]
-- if total_bal <= UINT64_MAX/2
-- if prev_bal * 3 >= total_bal * 2
-- if ~(curr_bal * 3 >= total_bal * 2)
```

`total_bal`의 계산에 사용되는
입력값 (상태, 블럭)의 필드 유래 추적

`b = state.VAL[0].EFFECTIVE_BALANCE`

↓ 입력 변이

`b' > UINT64_MAX/2`

앞에서 전체 잔액이 기준을 넘는 경우가 테스트되지 않은 것을 확인했으니, 이제 그 경우를 겨냥한 입력을 만들겠습니다.

여기서 total_bal은 입력에서 바로 바꿀 수 있는 필드가 아니고, 여러 잔액을 더한 값입니다. 따라서 실행 중 값의 출처를 추적해서, 이 합계가 입력의 어느 필드 왔는지를 찾습니다. 그리고 그 중 하나를 선택해 기준보다 큰 값으로 바꾸는 테스트를 만듭니다. 이 경우, 0번째 노드의 잔액을 기준값보다 크게 변이했습니다.

잔액은 모두 0 이상이므로, 잔액 하나만 기준을 넘겨도 조건을 어길 수 있게 되고, 앞서 말씀드린 불일치가 발견되었습니다. (45s)

결과: ① 기존 테스트가 놓친 조건을 검사했는가?

놓친 조건 126개를 추가로 검사

	(46.3%)	(**91.1%**)
조건 커버리지	130/281	→ 256/281 (+126)
파이썬 명세 분기 커버리지	152/358	→ 154/358 (+2)

126개 조건 중 **65개**는 원본 명세에 숨어있던 예외 조건

이렇게 생성한 테스트를 추가한 결과, 조건이 거짓인 경우의 커버리지는 약 46%에서 91%로 증가했습니다. 기존 테스트에서는 거짓 값이 나온 적 없던 조건 126개를 추가로 거짓으로 만들었다는 뜻입니다.
이 126개 중 65개는 원본 명세에 드러나지 않던 조건입니다. 명시화를 통해 이런 조건의 검사 여부까지 확인할 수 있게 되었습니다.

같은 테스트를 Python 명세에서 실행했을 때, 새롭게 실행된 분기는 두 개뿐이었습니다. 즉 분기 커버리지만으로는 어떤 조건을 추가로 검사했는지 파악하기 어렵다는 것을 보여줍니다. 저희는 명세에 조건을 드러내고 측정해서 이 차이를 진단할 수 있게 되었습니다. (40s)

결과: ② 실제 불일치를 발견했는가?

클라이언트 간 불일치 27개 발견

Category	Divergences
입력 조건 위반	11
정수 범위 초과 처리	7
정수 디코딩 오류	4
함수 호출 조건 위반	3
프로토콜 버전 선택 오류	1
계산 결과 불일치	1
	27

27개 중 22개는 명시화한 조건과 관련

생성한 테스트 2만여개(21,444)를 다섯 클라이언트에서 실행해, 클라이언트 간 불일치를 총 27개 사례로 정리했습니다.

동그라미 하나가 사례 하나이고, 원인에 따라 묶어 표시했습니다. 앞에서 본 정수 오버플로뿐만 아니라, 입력을 정수로 디코딩하는 과정에서 값이 달라지거나, 상태가 지켜야 할 조건을 위반한 입력을 서로 다르게 처리하는 사례도 발견했습니다.

27개 중 22개는 저희가 명세에 명시적으로 추가한 조건으로 인해 발견할 수 있었습니다. 앞에서 본 잔액 예시도 여기에 해당합니다.

다만 검증 설정에 따라 차이가 있습니다. 23개는 블록 제안자 시점에서만 불일치가 나타났고, 4개는 블록 제안 여부와 관련없이 불일치가 발생했습니다.

즉 명시화한 조건은 커버리지 수치만 늘린 것이 아니라, 실제로 구현들이 다르게 처리하는 입력을 찾게 해주었습니다. (60s)

결과: ③ 명세 정보가 도움이 되었는가?

같은 테스트 수에서 더 많은 불일치 발견

방법	발견한 불일치
무작위 값 변형	12
경계값 중심 변형	16
SpecTrum	27

각 방법에서 테스트 21,444개 생성

오버플로우 관련 불일치가 많다보니, 큰 값이나 경계값을 생성하는 것으로 충분할지도 모릅니다. 이를 확인하기 위해 같은 수의 테스트를 생성하는 두 방법과 비교했습니다.

입력 필드와 값을 무작위로 고르는 방법은 12개, 필드는 무작위로 고르되 경계값을 넣는 방법은 16개를 발견했습니다. 명세의 조건과 값의 출처를 활용한 SpecTrum은 27개를 발견했습니다.

다만 발견한 사례가 모두 겹치는 것은 아닙니다. 비교한 두 방법도 각각 SpecTrum이 놓친 사례 5개를 찾았습니다. 이 실험에서는 SpecTrum이 더 많은 사례를 찾았지만, SpecTrum은 581개의 spectests 중 59개만 선택해 시드로 활용하기 때문에 탐색 범위에 한계가 있다는 것도 볼 수 있습니다. (45s)

한계 Limitation

① 실제 네트워크에서의 악용 가능성은 미검증
동일 입력에 대한 처리 불일치만 확인

② 세 단계에 수동 작업이 필요
명세 작성: 예외 명시화, 조건 재구성
테스트 준비: 거짓이 될 수 없는 조건 제외

저희 결과에는 크게 두 가지 한계가 있습니다.
첫째, 주어진 입력에 대해 구현들이 다르게 처리한다는 사실만 확인했습니다. 이런 입력이 실제 네트워크의 실행 과정에서 등장할 수 있는지, 이를 이용해 공격할 수 있는지까지는 확인하지 않았습니다.
둘째, 아직 수동 작업이 많습니다. 숨어 있는 예외를 명시하고 조건을 규칙으로 재구성하는 작업뿐만 아니라, 달성 불가능한 커버리지 항목을 제외하고 테스트할 목표 조건을 정하는 작업도 사람이 수행했습니다. 대상인 Capella 명세의 규모가 작아 수동 작업이 감당 가능했으나, 더 큰 명세에 적용하려면 이 작업을 줄여야 할 것입니다. (45s)

향후 계획 Future Work

수동 작업을 자동화해 더 큰 명세(Gloas)에 적용

명세 작성
테스트 준비

숨어있는 예외 조건 자동 추출
규칙간 서로소/완전성 검사
→ 거짓이 될 수 없는 조건 자동 탐지

다음 단계는 이 수동 작업을 자동화해서 더 큰 명세인 Gloas에 적용하는 것입니다.
예외 조건을 드러내고 규칙을 재구성하는 작업, 그리고 커버리지와 테스트의 대상이 될 조건을 분류하는 작업에서 사람의 부담을 줄이려 합니다.
예외 조건을 드러내는 부분은, overflow, underflow, 인덱스 접근, div-by-zero 등 언어 수준의 예외 조건을 자동으로 추출할 수 있게 합니다.
최종 결과에 따라 규칙을 재구성하는 작업에서는, 규칙간 서로소/완전성 검사를 통해 사람의 노력을 줄입니다.
이렇게 하면 테스트 준비 과정에서 필요한, 거짓이 될 수 없는 조건을 제외하는 작업도 상당 부분 줄이거나 자동화할 수 있습니다.

이를 통해 이번에 Capella에 적용한 접근이 더 큰 명세에서도 효과적으로 적용되는지 검증할 계획입니다. (50 s)

명세의 조건을 드러내면 테스트의 빈틈을 겨냥할 수 있다

조건 명시화
↓
커버리지 측정
↓
미검사 조건을 겨냥한 테스트

조건 커버리지 46.3% → **91.1%**

불일치 **27**개 발견

요약하면, 이 연구의 핵심은 예외 조건과 조건 조합이 명확하게 보이도록 명세를 재작성하는 것입니다. 그러면 테스트가 어떤 경우를 놓쳤는지 측정하고, 그 경우를 겨냥한 입력을 만들 수 있습니다. 저희는 이를 통해 기존 테스트가 검사하지 않은 조건을 추가로 검사하고, 클라이언트 간 불일치 27개를 발견했습니다.
들어주셔서 감사합니다. (25s)